

FundSeeker 代码评审实验：7 份评审报告的对比分析

- 日期：2026-08-20
- 分析对象：本目录下 7 份评审报告（`2026-08-20-fs-run*.md`、`2026-08-20-fs-ctl*.md`）
- 状态：已有结论

1. 背景（阅读本文所需的前置信息）

1.1 被评审项目

被评审项目是一个名为 **FundSeeker** 的 Python 数据采集与分析系统：定期从基金公司、银行理财、东方财富（Eastmoney）、新浪等外部数据源抓取基金/理财产品的净值、持仓、行情数据，写入 PostgreSQL 数据库，并提供相似度聚类、归因等分析能力。系统通过 cron 脚本做周期性调度。本次评审主要涉及两类代码：

- **采集链路**：`src/fundseeker/runner.py`（批任务编排：循环产品列表、调用采集器、写库、登记运行日志）与 `src/fundseeker/utils/http.py`（带礼貌延迟和重试的 HTTP 客户端 `PoliteHttpClient`）及其上下游调用链；
- **相似度分析调度链路**：`scripts/fundseeker_similarity_cron.sh` 及其调用的聚类/归因持久化代码。

1.2 实验设置

这是一组用 **LLM 执行代码评审** 的对照实验，目的是观察"是否给评审 agent 提供结构化评审指引（SKILL）"对评审产出的影响：

- **run 组（实验组，有 SKILL 指引）**：评审者加载名为 `code-review` 的 skill（v5 版本）。该 skill 提供：①一组编号评审规则 **RR-1~RR-8**（契约一致性、变更登记、登记真实性、状态生命周期、幂等与重放、并发与锁、LLM 输出防御、外部系统输入防御、用户输入防御、前端一致性——评审时需逐条做适用性判定）；②统一的"评审意见卡"输出模板（每条意见含级别、类型、溯源、证据、维度、建议、分级依据）；③评审纪律要求。
- **ctl 组（对照组，无 SKILL 指引）**：同一模型在不加载该 skill 的条件下，仅凭通用代码评审能力完成同样的任务。

两组均遵守**隔离纪律**：评审对象是某个历史提交（commit）的代码快照，通过 `git show <commit>:<路径>` 读取；不允许读取"修复该状态缺陷的后续提交"的内容，也不允许读取既有的评审报告与修复记录——以避免后见之明污染评审结论。

意见级别约定：**P0** 最严重（必须立即修复），**P1** 严重，**P2** 中等，**P3** 轻微/风格类。

1.3 评审材料清单

被评审项目在时间上有这样一条修复链：每个提交都是"修复前一个状态的缺陷"：

```
517eb08 → c5ee3e3（给相似度 cron 脚本加 flock 防重入）
        → 05d66d3（修复 collect --holdings 卡在僵死 socket）
        → b3a0342（后续修复）
```

7 份报告锚定的代码状态（称为"基线"）与评审范围如下：

报告	组别	基线	评审范围	意见数
run1	有 SKILL	c5ee3e3（05d66d3 修复前）	采集链路（runner.py + http.py 及调用链）	16
run1b	有 SKILL	c5ee3e3（同 run1，冷启动重复）	同 run1	11
run2	有 SKILL	05d66d3（b3a0342 修复前）	采集链路	6
run3	有 SKILL	517eb08（c5ee3e3 修复前）	相似度 cron 调度链路（不同模块）	8
ctl1	无 SKILL	c5ee3e3	采集链路	16
ctl2	无 SKILL	c5ee3e3	采集链路	12
ctl3	无 SKILL	c5ee3e3	采集链路	15

分析口径提醒：这 7 次评审并非对同一代码状态的 7 次重复。严格的"同任务多轮"样本是：run 组内 **run1 vs run1b**（2 轮，同基线同范围），ctl 组 **ctl1/ctl2/ctl3**（3 轮，同基线同范围）；跨组对比只能建立在 c5ee3e3 子集上（run1/run1b vs ctl1/2/3）。run2（修复后的新状态）与 run3（不同模块）只能作为补充证据，下文在引用时会注明。

2. 问题一：同组多轮评审，有多少实质性差异

2.1 ctl 组（3 轮同一基线，最纯净的可重复性样本）

三轮评审**稳定命中**的**核心主题**只有**约 5 个**（按问题主题归并，不计措辞差异）：

核心主题	ctl1	ctl2	ctl3
HTTP 4xx 客户端错误与 5xx/超时不区分地一律重试	✓	✓	✓
<code>run_holdings</code> 产品循环内裸 <code>except Exception</code> 吞掉所有异常、无任何日志	✓	✓	✓
缺少单产品/单请求级硬性截止时间，socket 挂起可放大为整批卡死	✓	✓	✓
重试循环内重复执行节流延迟 <code>_enforce_delay</code> ，失败请求等待时间被放大	✓	✓	✓（变体）
HTTP 连接池无 TCP keepalive，半开连接无法被识别	✓	✓	—

长尾发现近乎随机抽样：每份报告 12–16 条意见，其中约一半是该轮独有。典型例子：

- ctl1 独有：`connect/read` 超时未拆分、SQL `INTERVAL` 写法绑定 PostgreSQL 方言、`_parse_market_date` 空字符串输入会崩溃等；
- ctl2 独有：`robots.txt` 拉取走 `urllib` 默认通道、完全绕过自建 `session` 与超时配置；采集循环串行无并发；数据库 `engine` 进程退出前未 `dispose` 等；
- ctl3 独有：`_LegacyRenegotiationAdapter` 子类未调用 `super().__init__()`，启用它的两个采集器在首次发请求时会抛 `AttributeError` 崩溃；`skip_existing` 用全库最大报告日期做过滤导致跨机构误过滤；持仓表冲突唯一键纳入不稳定的资产名称列等。（**经 §5 复核修正**：前两条“独有发现”均为错判——`adapter` 崩溃断言不成立，`skip_existing` 的过滤方向被理解反了，详见 §5.3。）

严重度定级明显不稳定，同一问题跨轮可漂移两个级别：

同一问题	ctl1	ctl2	ctl3
4xx 无差别重试	P0	P2	P0
裸 <code>except</code> 吞错	P0	P2	P1
无请求级总超时	P1	P1	P0

甚至出现事实判断对立的结论：

- ctl2 在“未发现问题”清单中写“`robots.txt` 解析失败时降级为放行是保守合理策略 ✓”；而 ctl1、ctl3 都把同一行为立为 P2 缺陷（fail-open 绕过 `robots` 守门）。
- ctl1 在“未发现问题”清单中写“`skip_existing` 仅排除最新季度已覆盖产品的逻辑与设计意图一致 ✓”；ctl3 却把同一段代码立为 P1 缺陷。后续 `run2` 在修复后的代码状态上独立确认了“该逻辑是缺陷”的方向（该过滤逻辑语义错误，会导致新季度发布后历史产品永远不被自动发现）

——即 **ctl1** 的“已核对无误”是一条被证伪的断言。（经 §5 复核补充：缺陷确实存在，但 **ctl3** 对其机理的描述——“跨机构误过滤”——方向反了，属“结论对、机理错”，见 §5.3。）

2.2 run 组 (run1 vs run1b, 同一基线)

两轮重合的主题**只有约 4 个**：裸 except 吞错、HTTP 层 socket 兜底缺失 (keepalive/超时)、`--holding-years` 数值参数无硬上限、robots 缓存缺陷。其余发现大量互斥：

- **run1b 独有的重头发现**：`run_holdings` 把多年多季度的持仓数据折叠进单一 `HoldingReport` 记录，导致外键指向的季度与明细行上的 `report_date` 列不一致、跨季度数据被唯一键静默折叠丢弃 (P1，数据完整性问题)。这是两轮 run 中最深的一条发现，**run1 完全漏掉**。run1b 另外独有：`skip_existing` 全局最大日期过滤不精确、采集器端 `years` 默认值 3 与 CLI/runner 端默认值 1 不一致、HTML 正则切片解析脆弱等。
- **run1 独有的运维面发现 (9 项)**：运行日志 `status="running"` 状态无 stale 回收机制（进程被杀后留下“伪运行”记录）；新增行数统计把冲突跳过的行也计入导致虚高；报告行已提交但明细失败时留下孤儿数据；`records_count` 把四种不同粒度的计数相加、语义不可解析；全部产品失败时仍返回 success 且进程退出码为 0；HTTP Session 无显式关闭；采集 cron 缺 flock（同仓库另一个 cron 已有此保护，形成对照）；配置文件中 `default_headers` 是无任何代码消费的死配置；User-Agent 硬编码等。

run 组内部同样存在一致性问题：

- 对 `_LegacyRenegotiationAdapter`（一个启用已知不安全 TLS 重协商的适配器），run1 在“未发现问题”清单中**称赞**其为“显式平台语义声明的好实践”，run1b 却把同一机制立为 P3 风险项（引入不安全开关但未在设计文档登记）。（经 §5 复核判定：run1 一方事实正确，run1b 该条为错判——两个采集器确实启用该开关，且 docs/ver1.0 已登记，详见 §5.3。）
- 规则适用性判断不一致：run1 以宽松方式启用了 RR-4（状态生命周期）并据此立了 stale 日志回收一条，run1b 则判定该规则不适用于此代码；
- 严重度漂移：`--holding-years` 无上限，run1 定 P2、run1b 定 P1。

2.3 问题一结论

两组组内多轮评审都存在**实质性差异**，不是“同一份报告换个措辞”。稳定重合的仅是少数高显著度问题（约 4–5 个主题）；以主题计，两轮之间的重合率大约只有 30–50%。差异体现在三个层面：

1. **覆盖点抽样**：长尾发现各轮互不相同，单轮评审更像“对缺陷空间的一次采样”；
2. **严重度定级**：同一条目跨轮可漂移两个级别；
3. **事实判断**：出现“无误”与“缺陷”的直接对立结论，且 **ctl** 组有一例被后续证据证伪。

实践含义：若要提高查全率，**多轮并行评审再归并**比优化单轮评审的提示词更直接有效——多轮之间的互补性很强。

3. 问题二：有无 SKILL 指引，结果质量是否存在可识别差异

3.1 覆盖驱动方式不同，产生了可验证的命中差异

run 组的评审由规则集驱动：逐条声明 RR-1~RR-8 的适用性判定（启用/不启用及理由），并按规则组织“未发现问题的检查项”清单。ctl 组是自由发挥，用“可靠性/性能/安全”等直觉维度打标签。这一机制差异带来了**系统性问题类别上的稳定命中差异**：

- **用户输入硬上限（RR-7c 覆盖的类别）**：`--holding-years` 数值参数无上限（传入大值可导致单产品数万次 HTTP 请求），run1、run1b 两轮都立项；**ctl 三轮全部漏掉**。run3 在另一条评审链路上还系统性挖出了 `--k`、`--top-n`、`--index-codes`、`--benchmark-code` 等一批同类缺口——这是规则强制“遍历所有用户输入面”才会有的产出形态。
- **契约与登记真实性（RR-1/RR-3 覆盖的类别）**：`records_count` 混合粒度、全部失败仍声明 success 且退出码为 0、`default_headers` 死配置、采集 cron 缺 flock——run1 独有，ctl 三轮全漏。这类“口径/登记”缺陷不是直觉式可靠性审查会自然触及的。
- 反向证据同样存在：ctl 组偏重的纯网络/资源层问题（如 ctl2 发现 robots.txt 拉取完全绕过自建 HTTP session、engine 未显式 dispose）run 组也未稳定命中。**SKILL 提升的是检查面的系统性，并不提升单点发现的深度。**

3.2 深度发现两组都有，SKILL 没有垄断

run 组最深的发现（多季度持仓折叠导致的数据完整性问题，出自 run1b）和 ctl 组最深的发现（TLS 适配器缺 `super().__init__()` 导致启用即崩溃，出自 ctl3）都是跨文件追踪的产物，且各自只在一个轮次中出现。两组也都漏掉了对方找到的深度发现。**深度发现更像单次采样的运气，与有无 SKILL 无关。**（经 §5 复核修正：ctl3 那条“最深发现”经核验为错判，不成立；run1b 的多季度折叠发现复核成立。修正后的图景是：本轮实验中唯一经复核确认的深度发现独属于 run 组——但样本只有 1 个，不足以推翻“深度发现是单轮采样运气”的判断，见 §5.6。）

3.3 定级口径与可审计性

- run 组每条意见带显式“分级依据”陈述，且整体定级克制（在 c5ee3e3 基线上最高只给到 P1）；ctl 组直接给出 P0（4xx 重试、裸 except 吞错均定 P0）且无依据说明。参照实际修复历史（05d66d3 真正修复的是 socket 僵死问题），run 组将其定 P1 的口径与实际修复优先级更吻合；但无论如何，**run 组的定级可回溯、可复核**，ctl 组不可。
- run 组报告结构高度统一（意见卡模板 + 问题类型汇总 + 规则集版本 + 隔离纪律声明 + 检查文件数等元信息）；ctl 组格式逐轮漂移（ctl1 接近模板、ctl2 简化、ctl3 又是另一种结构）。对“评审结果要归档、要跨轮比对、要驱动修复”的使用场景，**run 组产物的工程可用性明显更好。**

3.4 事实错误率

ctl 组出现两处与同组其他轮直接矛盾、或被后续证据证伪的"未发现问题"断言 (skip_existing 判无误、robots fail-open 判合理, 见 §2.1)。run 组有内部评价不一致 (TLS 适配器一轮褒一轮贬), 但未出现被后续证据证伪的硬性错误断言。样本量太小, 这只能作为方向性信号, 不足以下强结论。

(经 §5 复核修正: skip_existing 判无误确认为错误断言; robots fail-open 判合理经核验属可辩护的务实策略, 不计入错误。修正后 ctl 组的"未发现问题"错误断言为 1 处而非 2 处, 见 §5.3、§5.5。)

3.5 SKILL 没有降低组内方差

值得强调: run 组两轮的差异幅度 (重合仅约 4 个主题) 并不比 ctl 组 (重合约 5 个主题) 小。SKILL 规范化的是评审过程与覆盖面, 而不是输出的稳定性——多轮评审结果的随机性在两组中同样显著。

3.6 问题二结论

有无 SKILL 指引的差异可以识别, 主要体现在:

1. **过程规范化**: 规则适用性判定、统一模板、元信息、分级依据——run 组产物可审计、可归档、可跨轮比对;
2. **非直觉类别的系统性命中**: 用户输入防御、契约/登记一致性这类规则强制遍历的检查面, run 组稳定命中、ctl 组稳定遗漏;
3. 对核心可靠性缺陷的检出能力两组相当; 深度发现两组互有胜负; SKILL 未能压缩组内多轮的波动。

4. 总体判断与局限

总体判断:

1. 单轮 LLM 代码评审的产出应视为"对缺陷空间的一次采样": 核心高危问题稳定复现, 但约一半发现是单轮独有, 定级可漂移, 偶有对立结论。多轮并行 + 归并是提升查全率的直接手段。
2. SKILL 指引的价值不在"找到更深的 bug", 而在: ①把评审从直觉驱动变为规则驱动, 系统性覆盖非直觉检查面 (输入硬上限、契约/登记口径); ②产出结构化、可审计、可归档的评审文档。这些差异稳定可识别。
3. SKILL 指引未能解决: 单轮发现的随机性、深度发现的不确定性。

本分析的局限:

- run 组 4 份报告中只有 run1/run1b 是同基线重复（run2、run3 更换了基线与评审范围），因此 run 组内一致性结论的样本量仅为 2，统计强度弱于 ctl 组（n=3）；
- 跨组对比建立在 c5ee3e3 子集上（2 份 run vs 3 份 ctl），样本量小，§3.4 的错误率对比尤其只能视为方向性信号；
- 本分析只比较报告文本，未对每条意见做“是否真实缺陷”的逐条代码核验（已于 2026-08-20 补充完成逐条复核，见 §5）。

5. 复核：评审结果的错判分析（2026-08-20 补充）

5.1 复核方法

对 7 份报告的全部意见（合计 85 条）做逐条独立复核：以每份报告声明的基线 commit 为准，用 `git show <commit>:<路径>` 读取被评审代码快照，核对每条意见的事实性描述与代码实际行为是否一致。判定口径：

- **错判**：事实错误——代码不是报告所说的那样；
- **部分成立**：事实内核正确，但机制断言或影响量化明显失真；
- **不算错判**：事实正确、仅严重度/定性可争议（严重度失真单列说明）；行号引用漂移但代码事实可核实的，按定位瑕疵处理，不计错判。

此外，对三处跨报告结论冲突的断言（skip_existing 过滤逻辑、robots.txt fail-open、`_LegacyRenegotiationAdapter` 安全性）做了专项判定；多个复核过程相互独立且结论一致，其中 skip_existing 一条还有修复提交 b3a0342 的提交信息（自述 "new-quarter discovery" 修复）作为外部佐证。

5.2 复核结果总览

报告	意见数	错判	部分成立	漏报（召回率）*	严重度失真（不计错判）
run1	16	0	3	19 (46%)	#1/#5 定 P1 偏高、#12 定 P2 偏高
run1b	11	1	1	24 (31%)	P1-2 成本量化夸大
run2	6	0	1	无法评估*	P1-3 略偏高；3 条 P1 均被修复提交 b3a0342 独立证实
run3	8	0	3	无法评估*	F4 定 P1 偏高
run 合计	41	1 (2.4%)	8 (19.5%)	—	

报告	意见数	错判	部分成立	漏报（召回率）*	严重度失真（不计错判）
ctl1	16	0	1	21 (40%)	两条 P0 均偏高
ctl2	12	0	2	23 (34%)	P1-4 偏高
ctl3	16	2	2	22 (37%)	两条 P0 均偏高；报告自报"15 条"与正文 16 条不符
ctl 合计	44	2 (4.5%)	5 (11.4%)	—	

* **漏报统计口径**：仅适用于 c5ee3e3 基线、runner.py + http.py 范围的 5 份报告

(run1/run1b/ctl1/ctl2/ctl3)。将这 5 份报告中**经复核确认真实存在**的问题按主题去重归并（同一缺陷的不同侧面合并为一个主题，错判条目剔除），得到 **35 个确认问题主题**（HTTP/网络层 8 个、runner 逻辑层 26 个、CLI 输入 1 个）；某报告的"漏报数 = 35 - 该报告命中的主题数"，括号内为召回率（命中主题数 / 35）。ctl3 对 skip_existing 主题的机理描述错误 (§5.3)，但其确实把该代码立为缺陷，召回上计为命中。run2/run3 针对其他代码版本，现有复核数据无法确认其在各自基线上漏掉了什么，按实验约束不评估、不重跑复核。

另在"未发现问题"检查项中复核出错误断言 3 条：run1 一条次要（称 `--holding-code` 有 choices 限定，实际没有）；**ctl1、ctl2 各一条关键错误**——都把 `skip_existing` 全局最大日期过滤逻辑背书为无误，而该缺陷真实存在（见 §5.3）。错误背书是漏报的最严重形态：不仅没报，还明示"已检查、无问题"，阻断了后续人工或其他轮次补查的可能。

5.3 三处跨报告冲突断言的判定

- skip_existing 全局 MAX(report_date) 过滤：缺陷真实存在。** 该逻辑取全库最新报告日期，凡持有该日期报告的产品一律跳过。在收敛稳态（所有产品都采到当前最新季度）下，新季度发布后没有任何产品会被选中采集，全库最大值永远无法前进，新季度永远不被自动发现——除非恰有新产品入库打破僵局。修复提交 b3a0342 改为按产品粒度取最大值并 LEFT JOIN，独立证实该缺陷。**判定：ctl1 的"与设计意图一致 ✓"与 ctl2 的"不会漏新季度产品 ✓"均为错误背书（漏报）；run2 (P1-1) 与 run1b (P2-1) 判断正确；ctl3 的结论方向正确但机理描述错误——它声称"未采到新季度的其他机构产品会被误过滤"，而代码的排除方向恰好相反（未采新季度的产品不在排除集中，会被正常选中），属"结论对、机理错"。**
- robots.txt 解析失败 fail-open：真实存在但属可辩护的设计取舍。** 网络层失败时 `except Exception: return True` 属实；但 ctl3 升级为缺陷的关键论据"失败结果写入缓存、永久绕过 robots"为事实错误——缓存写入发生在 `read()` 成功之后，失败时下次请求会重新拉取。且 `RobotFileParser.read()` 本身对 4xx 响应按"允许"处理、对 401/403 按"拒绝"处理，网络不可达时放行是主流爬虫的常见务实策略。**判定：ctl2 的"合理"背书可辩护（仅"保守"一词不贴切）；ctl1/ctl3/run1b 立为缺陷属可争议观点，其中仅 ctl3 的缓存污染机理构成事实错误。**

3. `_LegacyRenegotiationAdapter` : `ctl3` 的 `AttributeError` 断言为明确错判。该类只覆写 `init_poolmanager` , 根本没有定义 `__init__` , 实例化时完整执行继承来的 `HTTPAdapter.__init__` (其内部多态调用被覆写的 `init_poolmanager`) , 不存在"子类未调 `super` 导致属性缺失"的路径; 复核中以基线同构代码实际实例化验证, 无任何异常。且 `cticwm`、`spdbwm` 两个采集器确实传了 `ssl_legacy=True` ——若断言为真, 这两个生产采集器必然首次请求即崩溃, 与项目后续正常演进矛盾。判定: **run1 的"好实践"评价事实无误; run1b 的 P3 ("无使用方、未登记") 为错判**——两个采集器实际启用该开关, 且 `docs/ver1.0` 有三处登记; `ctl1` 的 P2 ("无文档警示") 部分失实 (类 `docstring` 已写明 `unsafe`, `docs` 已登记, 仅参数级警示缺失) 。

5.4 错判与失真的原因分类

复核出的 3 条错判与 13 条部分成立, 按成因可归为五类:

1. **对语言/框架运行时行为的臆测** (`ctl3` 的 `adapter` 崩溃断言): 凭"子类没调 `super().__init__()` 会崩"的常见模式做模式匹配, 却没有核实最基本的 Python 语义——子类压根没定义 `__init__` , 继承链完整, "没调 `super`"的前提不存在。这是最典型的一类 LLM 评审幻觉: 从代码形状联想已知缺陷模式, 跳过实际验证。
2. **逻辑极性理解反转** (`ctl3` 的 `skip_existing` 误过滤): `~ProductInfo.id.in_(reported_ids)` 的否定方向被理解反, 把"排除已有最新报告的产品"读成"排除未采新季度的产品"。
3. **机制链条单环失真导致影响放大** (13 条部分成立的主体, `run1 #1/#5/#9`、`run1b P1-2`、`run2 P2-1`、`run3 F3/F4/F6`、`ctl1 P2`、`ctl2 P1-4/P2-2`、`ctl3 robots` 缓存): 对 `requests/urllib3` 超时与重试行为、PostgreSQL MVCC 可见性、HTTP 请求频次等机制细节的断言不准, 后果被数倍夸大 (如把 `4xx` 快速响应错套 `30s` 读超时算出 "152s/URL"; 把每年 1 次请求算成每年 4 次)。代码观察是对的, 推演是糙的。
4. **隔离纪律下的越界断言** (`run1b P3-3`、`ctl1 P2` 部分): 评审协议禁止读取 `docs/` , 评审者却对"设计文档是否登记"下了断言——而 `docs/ver1.0` 实际已登记。**这是评审协议本身的副作用, 与有无 SKILL 无关**: 隔离纪律防住了后见之明, 也制造了一类"对自己无权读取的材料做事实断言"的错误。
5. **计数与量化粗糙** (`run2` 的 4 倍请求量、`ctl3` 的"至少 3 次 `commit`"实为 2 次、`ctl3` 自报条目数与正文不符): 不影响定性, 但损害报告的可信度。

另有一个跨报告的普遍现象 (不计错判): **行号引用系统性漂移**, `run1/run1b/run2/run3` 多处的引用行号低于实际 40–210 行, 但所引代码内容逐字准确——推测评审时按分块或非基线视图定位行号。对消费报告的读者, 这增加了按图索骥的成本。

5.5 有无 SKILL 在错判方面的差异

- **错判率都低, 差异不显著**: `run` 组 1/41 (2.4%) , `ctl` 组 2/44 (4.5%) , 且 `ctl` 的 2 条都集中在 `ctl3` 一份报告。样本太小, 不能得出"SKILL 降低错判率"的结论。

- **失真形态不同**：run 组的 8 条部分成立几乎全是“机制推测失真/影响夸大”——规则驱动它覆盖更多“范围缺口”类条目，推测性机制断言也随之增多；ctl 组的错判则是两条**事实性硬错**（运行时行为臆测、逻辑极性反转）。方向上，SKILL 组的错误更“浅”（观察对、推演糙），对照组的错误更“硬”（观察本身就错）——同样因样本小，只能视为方向性信号。
- **严重度定级：复核支持了 §3.3 的方向判断**。ctl 组 4 个 P0 (ctl1、ctl3 各 2 个) 经复核全部认定偏高——这是一个内部 cron 批处理系统，无数据损坏或资损路径，常规定级 P1 更合理；run 组唯一的 P0 (run3 的 cron 无重入防护) 有真实事故 (2026-07-17 撞库 22 分钟) 背书，定级成立。SKILL 要求陈述“分级依据”的要求对过度定级有可见的抑制作用。
- **“未发现问题”背书的质量**：关键错误背书 run 组 0 条、ctl 组 2 条 (skip_existing)。这一条与 §3.1 呼应：规则清单强制 run 组把 skip_existing 这类逻辑当作检查对象 (run1b/run2 都立项了)，而不是在“已核对无误”里放掉。
- **评审协议的共性问题**：隔离纪律导致的越界断言 (§5.4 第 4 类) 在两组各出现 1 次，属实验设计层面应修的问题——建议评审协议明确“禁止对被隔离材料的内容做任何断言，只允许声明‘未核查’”。

5.6 复核对前文结论的修正

1. **§2.1/§3.2 修正**：ctl3 的“adapter 崩溃”发现是错判，“全部 7 份报告中最尖锐的单个发现”一说撤销。修正后，本轮实验中唯一经复核确认的深度发现是 run1b 的多季度持仓折叠（数据完整性缺陷，复核成立）——深度发现实际上独属于 run 组。但样本只有 1 个，前文“深度发现像单轮采样运气”的判断维持不变。
2. **§2.2/§5.3 修正**：run1 与 run1b 对 TLS 适配器的“一褒一贬”已判定——run1 正确，run1b 错判。此例从“内部不一致”改判为“run 组内部也出现事实性错判（1 条）”。
3. **§3.4 修正**：ctl 组的“未发现问题”错误断言由 2 处核减为 1 处 (robots fail-open 背书属可辩护)，但该处 (skip_existing) 是 7 份报告中最严重的一条错误背书——它背书的正是被修复提交证实存在的真实缺陷。
4. **总体判断维持**：复核没有推翻 §4 的三条总体判断，反而加强了第 1 条——多轮归并的价值不仅是查全（互补），还包括**纠错**：ctl3 的两条硬错判、ctl1/ctl2 的错误背书，都是在跨轮/跨组对照中才暴露出来的。单份报告的“未发现问题”清单和深度发现一样，都需要对照验证后才可采信。

5.7 漏报情况分析

以 35 个确认主题为分母，5 份 c5ee3e3 报告的单轮召回率仅 31%–46%——**每轮都漏掉了半数以上事后可确认的问题**，这为 §2.3“单轮评审是对缺陷空间的一次采样”提供了量化支撑。漏报呈现以下明显特征：

1. **高严重度缺陷的单轮检出并不可靠**。确认主题中最深的三条缺陷各自只有约 1/5 的轮次命中：多季度持仓折叠（数据完整性，P1 级）仅 run1b 报出；skip_existing 新季度停摆（P1 级，被

修复提交证实) 仅 run1b 与 ctl3 报出 (ctl3 机理还报错了), run1 未提及、ctl1/ctl2 更是错误背书; stale running 日志无回收仅 run1 报出。真正达到 4/5 以上命中率只有"显眼"的问题: 裸 except 吞错 (5/5)、4xx 无差别重试 (4/5)。**问题越深、越需要跨文件推理, 单轮命中越依赖运气**——深度发现与漏报是同一枚硬币的两面。

2. **漏报有系统性盲区, 不只是随机抽样。**最清晰的一例: `--holding-years` 无硬上限, run 组两轮全中、ctl 组三轮全漏——这是 SKILL 规则 (RR-7c 强制遍历用户输入面) 直接消除的盲区, 与 §3.1 的结论互为印证。反向盲区也存在: 重试循环内 `_enforce_delay` 节流叠加, ctl 组三轮全中、run 组两轮全漏。即**两组各有稳定的盲区, 但 ctl 组的盲区恰好覆盖规则强制检查的非直觉类别**。
3. **组间召回率差距不大, 但达到同等覆盖所需轮次不同。**ctl 组三轮的并集召回为 22/35 (63%); run 组仅用两轮就达到同样的 22/35 (63%)。换言之, 在本实验的规模下, **SKILL 指引约等于白送一轮评审的覆盖量**。
4. **高召回不等于高纵深。**run1 召回率最高 (46%), 其增量主要来自运维面 P2/P3 条目 (stale 日志、死配置、计数口径等), 但它漏掉了两条最深的缺陷 (多季度折叠、skip_existing); run1b 召回率最低 (31%), 却贡献了最深的发现。评估单轮质量时, 召回率与发现深度是两个独立的维度。
5. **错误背书是漏报的最严重形态** (见 §5.2 脚注): ctl1/ctl2 对 skip_existing 的"已核对无误"比单纯漏报危害更大——它给后续读者传递了虚假的安全感。run 组未出现关键错误背书。

口径局限: 分母 35 个主题是"被至少一轮发现且复核确认"的问题并集, **所有轮次都漏掉的问题 (未知的未知) 不在分母内**, 真实召回率只会更低; 主题归并的粒度 (如"超时配置"合并 connect/read 拆分与请求级 deadline 两个侧面) 会影响个位数级别的计数, 但不影响上述特征结论。